CampOS - Comprehensive System Design Document

Version: 2.0 **Date:** November 5, 2025 **Status:** Production (Current State) + Architectural Vision (Future State) **Document Type:** System Design & Transition Plan

---

Table of Contents

Executive Summary

Project Overview

CampOS is a multi-tenant SaaS platform for campground management, handling reservations, site management, payments, and property operations for independent campground owners.

Current State (v1.0)

- **Architecture:** Next.js 15 monolith with App Router
- **API:** 33 REST endpoints using Next.js API routes
- **Database:** Supabase PostgreSQL with RLS
- **Frontend:** React 19 with Server Components
- **Payments:** Stripe Connect for multi-tenant payments
- **Deployment:** Vercel serverless platform
- **Status:** Production-ready with 13 core tables, 40+ indexes

Future State (v2.0 - Modular Monolith)

- **Architecture:** Modular monolith with 8 bounded contexts
- **Communication:** Internal domain events + message bus
- **Database:** Logical separation with shared physical database
- **API:** Domain-driven design with clear module boundaries
- **ML/AI Integration:** Python-based ML service for intelligent automation
- **Scalability:** Prepared for microservices extraction if needed
- **Timeline:** 6-12 month phased transition (9-12 months with ML/AI)

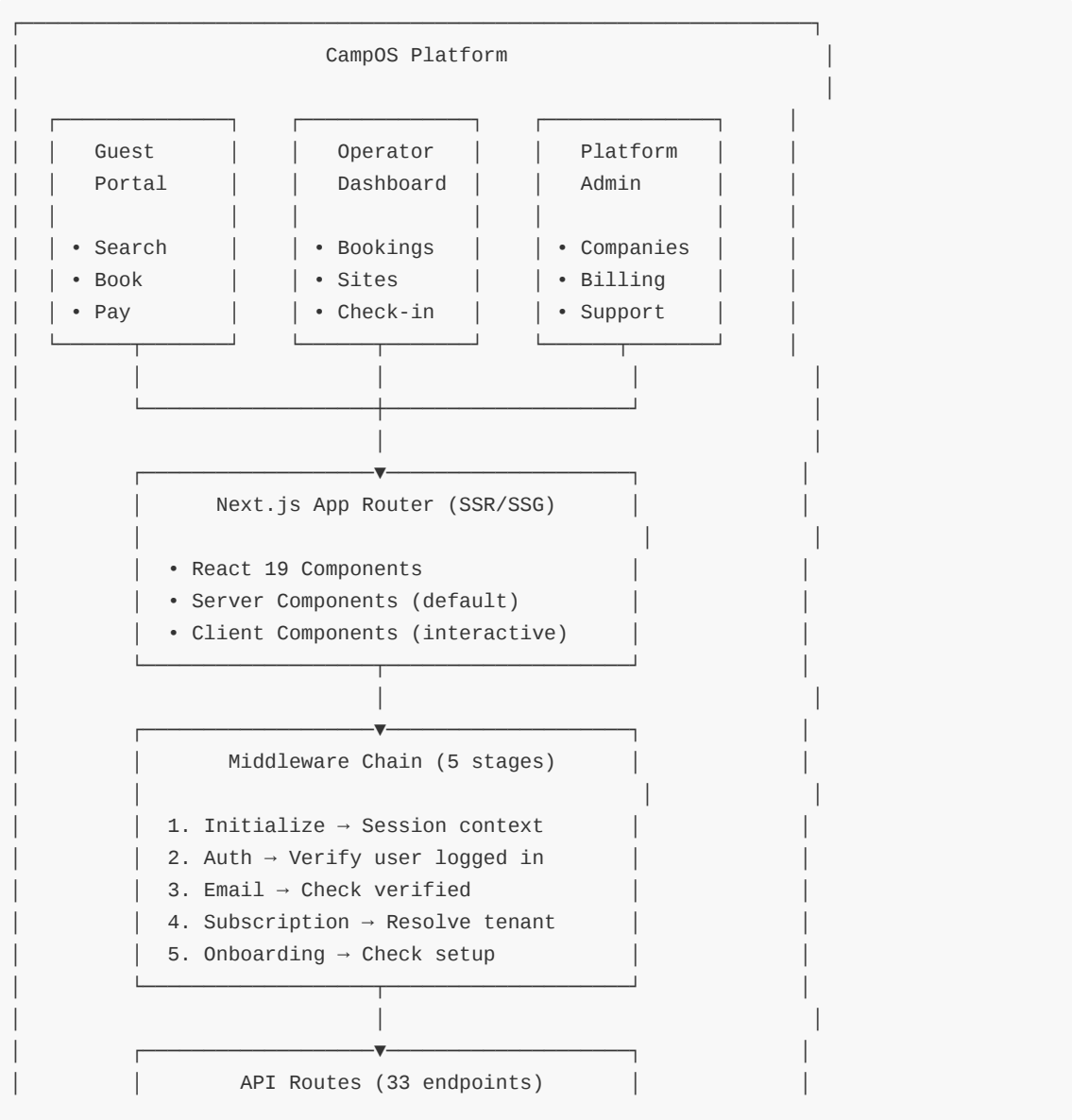
Key Metrics

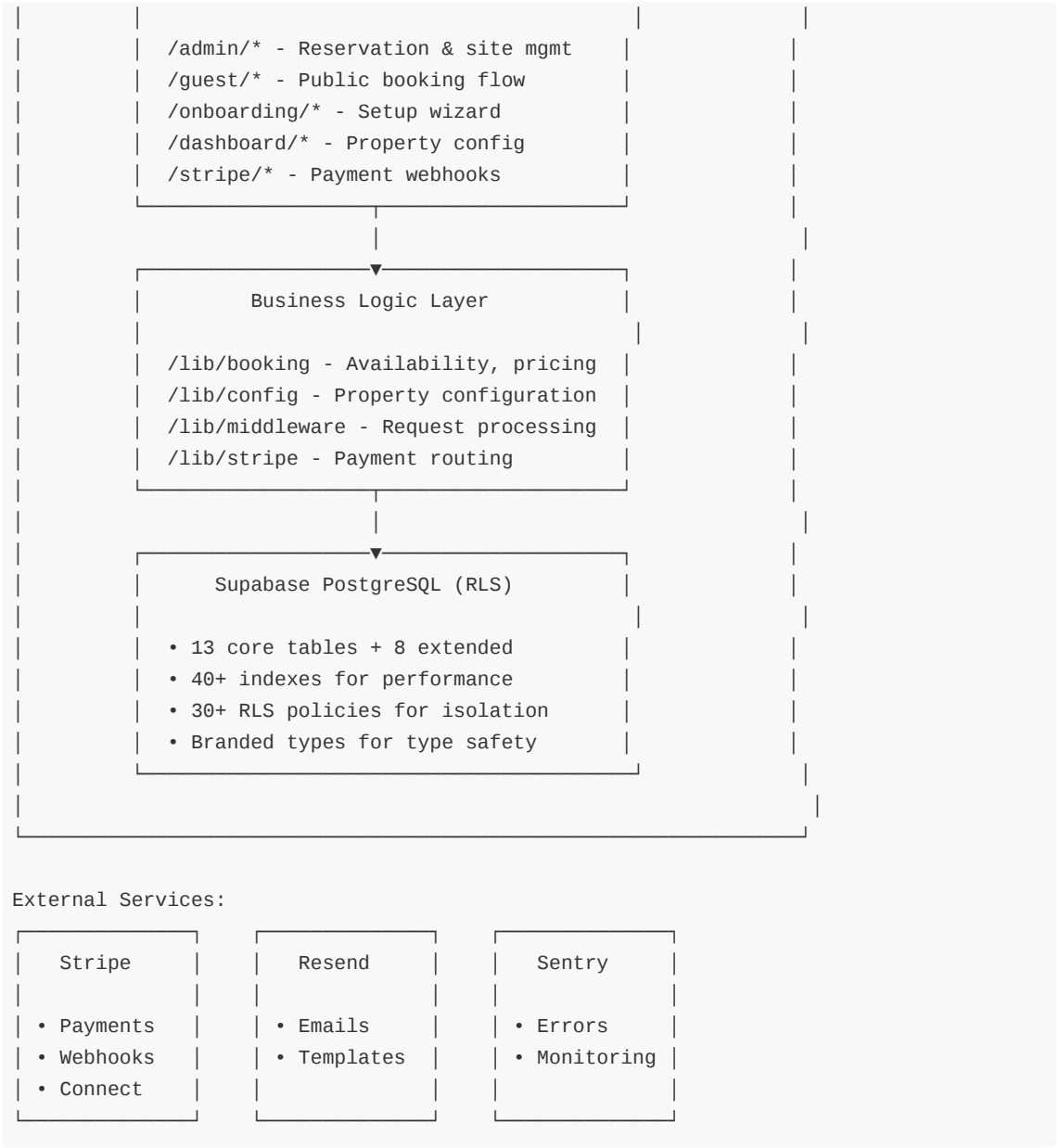
Metric	Current State	Future State
API Endpoints	33 endpoints	60+ domain endpoints (including ML)
Database Tables	13 core + 8 extended	Same + 7 ML tables
Modules/Services	Implicit (by directory)	8 explicit bounded contexts

Code Organization	Feature-based folders	Domain-driven modules
Testing Strategy	Unit + Integration + E2E	+ Contract + ML model testing
Deployment	Single Vercel deployment	Modular with feature flags + ML service
Scalability	Vertical (Vercel limits)	Horizontal (module extraction)
Intelligence	Rule-based	ML-powered optimization

Current State Architecture

1. System Context Diagram





2. Current Architecture Details

2.1 Frontend Architecture

Technology Stack:

- **Framework:** Next.js 15 (App Router)
- **UI Library:** React 19
- **Component Library:** Shadcn/UI (built on Radix UI)
- **Styling:** TailwindCSS (utility-first)
- **Forms:** React Hook Form + Zod validation
- **State:** Component-local state (no Redux/Zustand)

Component Patterns:

```
Server Components (Default):
- Data fetching at the server
```

```
- Better SEO and performance
- Direct database access via Supabase

Client Components ('use client'):
- Interactive UI elements
- Form handling
- Real-time updates
- Browser APIs
```

Route Structure:

```
/app
  /(auth)         - Public authentication
  /(marketing)    - Landing pages
  /dashboard      - Protected operator UI
  /onboarding     - Setup wizard
  /book           - Guest booking flow
  /api            - Backend API routes
```

2.2 Backend Architecture

API Design:

- **Pattern:** RESTful with JSON payloads
- **Routes:** 33 endpoints across 8 categories
- **Validation:** Zod schemas on all inputs
- **Authentication:** Supabase session cookies
- **Authorization:** Multi-tenant isolation via owner_id

Middleware Pipeline:

```
// Five-stage composition
Request → Initialize → Auth → EmailVerify → Subscription → Onboarding → Route

Stage 1: Initialize
  - Create session ID
  - Extract pathname, searchParams, origin
  - Initialize Supabase client

Stage 2: Authentication
  - Verify session via supabase.auth.getUser()
  - Add AuthContext: { userId, email, emailVerified }
  - Early exit on public routes (/api/guest/*, /api/webhooks/*)

Stage 3: Email Verification
  - Check email_confirmed_at !== null
  - Redirect to /verify-email if unverified
  - Required for /dashboard/* routes

Stage 4: Subscription & Tenant Resolution
  - Query companies WHERE owner_id = userId
  - Validate subscription_status = 'active'
  - Add TenantContext: { companyId, subscriptionStatus }
  - Block access if past_due or canceled
```

```
Stage 5: Onboarding Validation
  - Check property setup complete
  - Redirect to /onboarding if incomplete
  - Exception: /onboarding/* routes bypass this check
```

Business Logic Organization:

```
/lib
  /booking        - Availability, pricing, reservations
  /config         - Property configuration system
  /middleware     - Request processing pipeline
  /stripe         - Stripe Connect payment routing
  /email          - Email delivery via Resend
  /monitoring     - Sentry error tracking
  /supabase       - Database clients (browser, server, service-role)
  /csv            - Site import from CSV
  /analytics      - Dashboard analytics queries
  /constants      - Application constants
```

2.3 Database Architecture

Core Entities:

```
companies (1) ──→ (Many) properties
properties (1) ──→ (Many) sites, guests, reservations, seasonal_pricing_templates
sites (1) ──→ (Many) reservations
reservations (1) ──→ (Many) payments, reservation_actions, payment_installments
guests ←──→ reservations (Many-to-Many via FK)
```

Multi-Tenant Isolation:

- **Primary:** owner_id in properties table
- **Secondary:** property_id in all child tables
- **Enforcement:** RLS policies at database level + application queries
- **Type Safety:** Branded types (CompanyId, PropertyId, SiteId) prevent mixing

Configuration System:

- **Property-level defaults:** deposit, pricing, booking rules, rate discounts
- **Site-level overrides:** Per-site customization
- **Precedence:** Site override > Property default
- **Storage:** JSONB columns for flexibility

2.4 Payment Architecture

Stripe Integration:

```
Platform Account (CampOS)
  ↓
Connected Accounts (Campground Owners)
  ↓
Guest Payments → Direct to Connected Account

Platform never touches money (reduces liability)
```

```
Funds flow: Guest → Stripe → Campground Owner
Platform tracks: Subscription fees only
```

Payment Flow:

```
1. Guest books site
2. Create PaymentIntent via connected account
3. Guest pays via Stripe PaymentElement
4. Webhook: payment_intent.succeeded
5. Reservation status → confirmed
6. Confirmation email sent
```

Webhook Events Handled:

1. `checkout.session.completed` - Subscription signup
2. `customer.subscription.*` - Subscription lifecycle
3. `invoice.payment_*` - Billing events
4. `payment_intent.succeeded` - Guest booking confirmation

2.5 Security Architecture

Authentication:

- Supabase magic links (email-based, no passwords)
- Session tokens in httpOnly secure cookies
- Automatic token refresh via middleware
- Email verification gate for dashboard access

Authorization:

- Row Level Security (RLS) at database
- Application-level tenant filters
- Service role for admin operations only
- Branded types prevent ID confusion

Data Protection:

- All money in integer cents (no floating-point errors)
- Input validation via Zod schemas
- SQL injection prevention (parameterized queries)
- XSS prevention (React auto-escaping)
- CSRF protection (SameSite cookies + state parameters)

3. Current Pain Points

3.1 Critical Issues

1. Webhook Race Conditions:

```
Problem: checkout.session.completed and customer.subscription.created
         fire simultaneously from Stripe

Impact: Subscription setup fails silently
        Company creation takes 200-500ms
        Subscription update runs before company exists

Current Fix: Retry with fixed delays (insufficient)
```

```
Root Cause: No event ordering guarantees
            No idempotency protection
            No database transaction support

Solution Needed: Event store with ordering
                 Idempotency keys
                 Database transactions
```

2. Missing Idempotency:

```
Risk: Stripe retries webhook, creates duplicate data

Current: No idempotency key checking
         subscription_events table exists but not used for dedup

Needed: Store stripe_event_id, reject duplicates
```

3. Tight Coupling in API Routes:

```
Current: Route handler mixes:
         - Request validation
         - Business logic
         - Database queries
         - Response formatting

Better: Extract service layer:
        Route → Service → Repository pattern
```

3.2 Architectural Limitations

Scalability Concerns:

- Middleware hits database on every request
- No caching for subscription/tenant status
- Query N+1 problems in list operations
- CSV import not batched (slow with 100+ sites)

Missing Features:

- Multi-company management (MVP = 1 property per user)
- Staff role access (table exists, not implemented)
- Guest self-service portal
- Reservation modification by guests
- Analytics caching (queries hit live database)

Code Quality Issues:

- Pricing engine fragmented across files
- Configuration merging logic could be centralized
- Some duplicate validation logic
- Missing database transactions

Future State: Modular Monolith Architecture

1. Architecture Vision

Why Modular Monolith?

The modular monolith provides the benefits of microservices (loose coupling, clear boundaries, independent development) while avoiding the operational complexity (deployment coordination, distributed transactions, network latency).

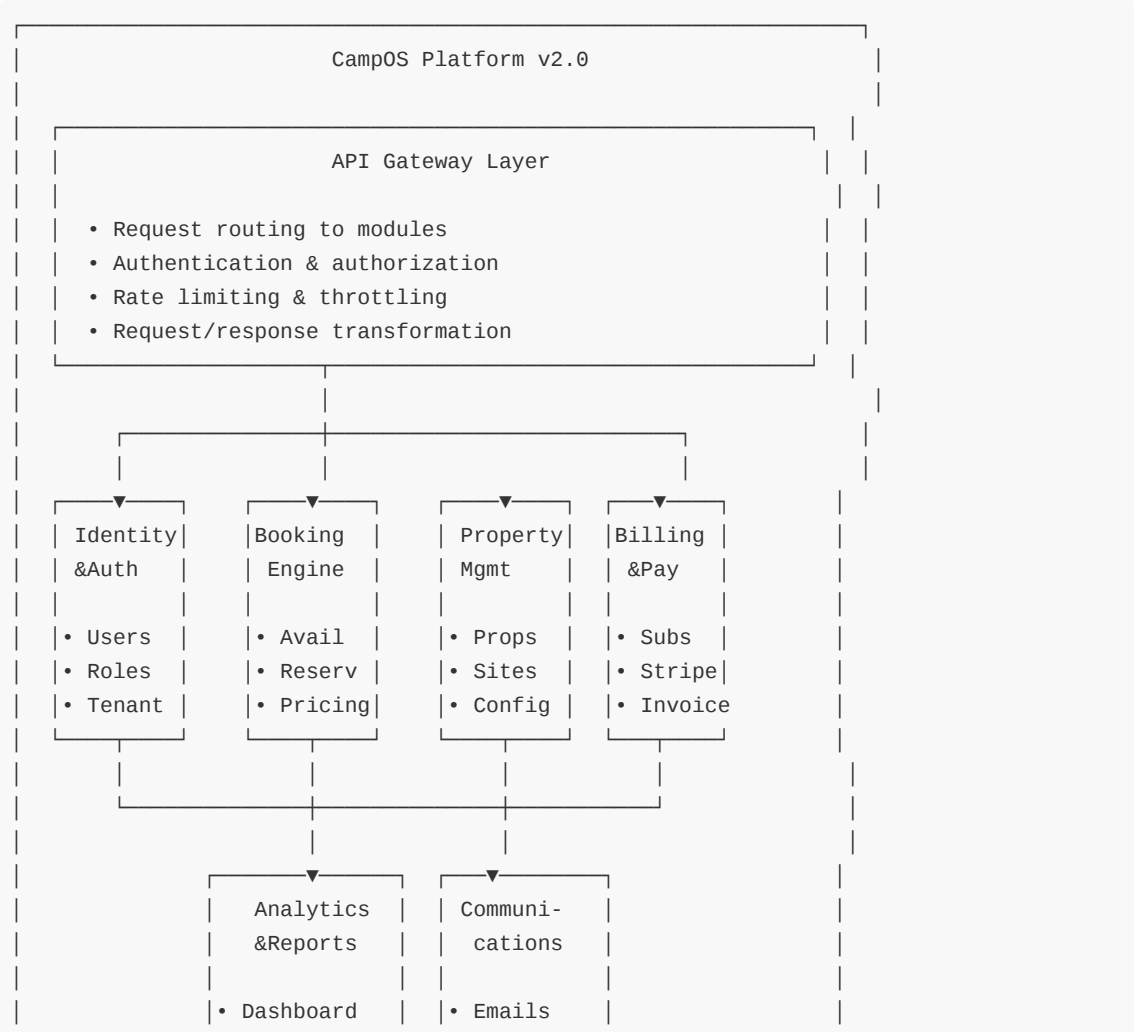
Key Principles:

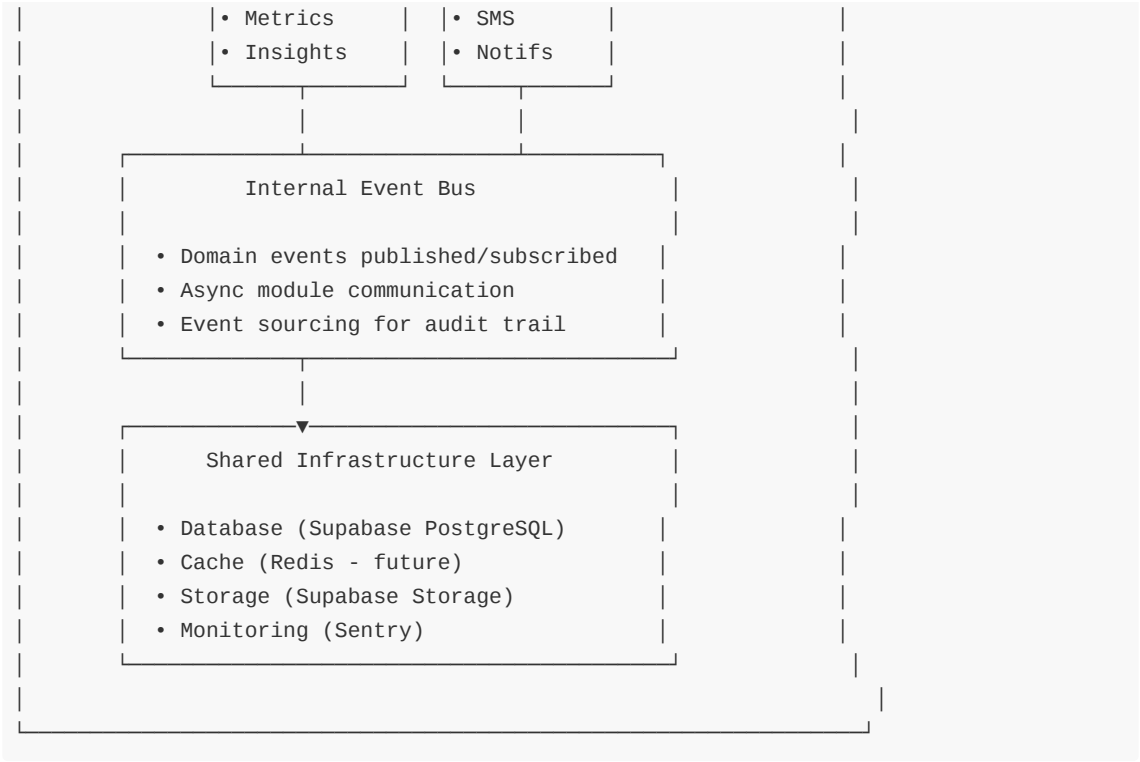
1. **Bounded Contexts:** Clear domain boundaries based on DDD
2. **Explicit Dependencies:** Modules declare what they need
3. **Event-Driven Communication:** Async internal messaging
4. **Shared Database with Logical Separation:** Tables grouped by domain
5. **Deployment Flexibility:** Can extract modules to microservices later
6. **Test Independence:** Each module fully testable in isolation

2. Module Architecture

2.1 Bounded Contexts (8 Modules)

Note: Module 8 (ML/AI & Intelligence) is documented separately in `SYSTEM_DESIGN_ML_AI_MODULE.md` due to its extensive scope and different technology stack (Python-based ML service).





2.2 Module Definitions

Module 1: Identity & Access Management

Responsibilities:

- User authentication (magic links, sessions)
- User registration and profile management
- Email verification
- Multi-tenant context resolution
- Role-based access control (RBAC)
- Permission management
- Staff user management

Database Tables:

- auth.users (Supabase managed)
- companies
- property_staff

API Endpoints:

- POST /auth/login
- POST /auth/verify-token
- GET /auth/session
- POST /auth/logout
- GET /tenant (subdomain resolution)

Domain Events Published:

- UserRegistered
- UserLoggedIn
- UserLoggedOut
- TenantResolved

- StaffMemberAdded
- StaffMemberRemoved

Dependencies:

- None (foundational module)

Module Interface:

```
interface IdentityModule {
  // Authentication
  login(email: string): Promise<MagicLinkSent>
  verifyToken(token: string): Promise<AuthSession>
  logout(sessionId: SessionId): Promise<void>

  // User management
  getUser(userId: UserId): Promise<User>
  updateProfile(userId: UserId, data: ProfileData): Promise<User>

  // Tenant resolution
  resolveTenant(subdomain: string): Promise<TenantContext>
  getTenantByUser(userId: UserId): Promise<TenantContext>

  // Authorization
  hasPermission(userId: UserId, permission: Permission): Promise<boolean>
  getUserRoles(userId: UserId): Promise<Role[]>

  // Staff management
  addStaffMember(propertyId: PropertyId, email: string, role: StaffRole):
Promise<void>
  removeStaffMember(staffId: StaffId): Promise<void>
}
```

Module 2: Booking Engine

Responsibilities:

- Availability search and checking
- Pricing calculation (base, weekend, seasonal, discounts)
- Reservation lifecycle (pending → confirmed → checked-in → checked-out)
- Guest management
- Check-in/check-out workflows
- Reservation modifications (extend, renew, modify, cancel)
- Booking rules validation

Database Tables:

- reservations
- guests
- reservation_actions
- payment_installments

API Endpoints:

- POST /booking/search-availability
- POST /booking/reserve (create pending reservation)

- POST /admin/reservations/create (manual booking)
- GET /admin/reservations/:id
- PATCH /admin/reservations/:id/update
- POST /admin/reservations/:id/check-in
- POST /admin/reservations/:id/checkout
- POST /admin/reservations/:id/cancel
- POST /admin/reservations/:id/actions (extend/renew)

Domain Events Published:

- ReservationCreated
- ReservationConfirmed
- ReservationCancelled
- GuestCheckedIn
- GuestCheckedOut
- ReservationExtended
- ReservationRenewed
- ReservationModified

Domain Events Subscribed:

- PaymentConfirmed (from Billing module)
- SiteStatusChanged (from Property Management)

Dependencies:

- Identity (tenant context, user info)
- Property Management (site availability, pricing config)
- Billing (payment status)

Module Interface:

```
interface BookingModule {
  // Availability
  searchAvailability(criteria: AvailabilityCriteria): Promise<AvailableSite[]>
  checkAvailability(siteId: SiteId, dates: DateRange): Promise<boolean>

  // Pricing
  calculatePrice(siteId: SiteId, dates: DateRange, guests: GuestCount):
Promise<PriceBreakdown>
  applyDiscount(price: Money, discountCode: string): Promise<Money>

  // Reservations
  createReservation(data: ReservationData): Promise<Reservation>
  confirmReservation(reservationId: ReservationId): Promise<Reservation>
  cancelReservation(reservationId: ReservationId, reason: string): Promise<void>

  // Lifecycle
  checkIn(reservationId: ReservationId, data: CheckInData): Promise<void>
  checkOut(reservationId: ReservationId, data: CheckOutData): Promise<void>

  // Modifications
  extendReservation(reservationId: ReservationId, newCheckout: Date):
Promise<Reservation>
  renewReservation(reservationId: ReservationId): Promise<Reservation>
```

```
  // Queries
  getReservation(reservationId: ReservationId): Promise<Reservation>
  getReservationsByProperty(propertyId: PropertyId, filters: Filters):
Promise<Reservation[]>
  getUpcomingCheckIns(propertyId: PropertyId, date: Date): Promise<Reservation[]>
}
```

Module 3: Property Management

Responsibilities:

- Property (campground) CRUD
- Site (campsite) CRUD
- Property configuration (deposit, pricing, booking rules)
- Site status management (available, maintenance, housekeeping)
- Amenities and features management
- Seasonal pricing templates
- CSV bulk site import
- Onboarding wizard

Database Tables:

- properties
- sites
- seasonal_pricing_templates
- site_seasonal_template_applications

API Endpoints:

- GET /properties
- POST /properties (create)
- GET /properties/:id
- PATCH /properties/:id
- DELETE /properties/:id
- GET /properties/:id/settings
- PATCH /properties/:id/settings
- POST /properties/:id/sites (bulk CSV or single)
- GET /sites
- GET /sites/:id
- PATCH /sites/:id
- DELETE /sites/:id
- POST /sites/:id/housekeeping-complete
- POST /onboarding/setup-property
- POST /onboarding/add-site
- POST /onboarding/complete

Domain Events Published:

- PropertyCreated
- PropertyUpdated
- PropertyDeleted
- SiteCreated
- SiteUpdated
- SiteDeleted
- SiteStatusChanged

- ConfigurationUpdated
- OnboardingCompleted

Domain Events Subscribed:

- GuestCheckedIn (update site.status = occupied)
- GuestCheckedOut (update site.status = housekeeping)
- ReservationCancelled (update site.status = available)

Dependencies:

- Identity (tenant context, ownership validation)

Module Interface:

```
interface PropertyModule {
  // Property CRUD
  createProperty(data: PropertyData): Promise<Property>
  getProperty(propertyId: PropertyId): Promise<Property>
  updateProperty(propertyId: PropertyId, data: Partial<PropertyData>):
Promise<Property>
  deleteProperty(propertyId: PropertyId): Promise<void>

  // Site CRUD
  createSite(propertyId: PropertyId, data: SiteData): Promise<Site>
  bulkImportSites(propertyId: PropertyId, csvData: string): Promise<BulkImportResult>
  getSite(siteId: SiteId): Promise<Site>
  updateSite(siteId: SiteId, data: Partial<SiteData>): Promise<Site>
  deleteSite(siteId: SiteId): Promise<void>

  // Site status
  updateSiteStatus(siteId: SiteId, status: SiteStatus): Promise<void>
  markHousekeepingComplete(siteId: SiteId): Promise<void>

  // Configuration
  getConfiguration(propertyId: PropertyId): Promise<PropertyConfiguration>
  updateConfiguration(propertyId: PropertyId, config: Partial<PropertyConfiguration>):
Promise<void>

  // Queries
  getSitesByProperty(propertyId: PropertyId, filters: Filters): Promise<Site[]>
  getAvailableSites(propertyId: PropertyId, dates: DateRange): Promise<Site[]>
}
```

Module 4: Billing & Payments

Responsibilities:

- Stripe subscription management
- Stripe Connect OAuth
- Payment processing (guest bookings)
- Invoice generation
- Webhook handling (Stripe events)
- Payment installment tracking
- Refund processing

Database Tables:

- payments
- subscription_events
- payment_installments

API Endpoints:

- POST /stripe/create-checkout
- GET /stripe/verify-session
- GET /stripe/connect/authorize
- POST /stripe/webhook
- POST /booking/create-payment-intent
- POST /guest/payment/confirm
- POST /dashboard/properties/:id/stripe-disconnect

Domain Events Published:

- PaymentConfirmed
- PaymentFailed
- SubscriptionCreated
- SubscriptionUpdated
- SubscriptionCancelled
- InvoicePaid
- InvoiceFailed
- RefundIssued

Domain Events Subscribed:

- ReservationCreated (create payment intent)
- ReservationCancelled (process refund)

Dependencies:

- Identity (company/property for Stripe routing)
- Booking (reservation context for payments)

Module Interface:

```
interface BillingModule {
  // Subscriptions
  createCheckoutSession(plan: PlanId, companyData: CompanyData):
Promise<CheckoutSession>
  verifyCheckoutSession(sessionId: string): Promise<SubscriptionVerification>
  cancelSubscription(companyId: CompanyId): Promise<void>

  // Stripe Connect
  authorizeConnectedAccount(code: string, propertyId: PropertyId): Promise<void>
  disconnectStripeAccount(propertyId: PropertyId): Promise<void>

  // Payments
  createPaymentIntent(reservationId: ReservationId): Promise<PaymentIntent>
  confirmPayment(paymentIntentId: string): Promise<Payment>
  processRefund(paymentId: PaymentId, amount: Money): Promise<Refund>

  // Webhooks
  handleWebhookEvent(event: StripeEvent): Promise<void>
```

```
  // Queries
  getPaymentHistory(propertyId: PropertyId): Promise<Payment[]>
  getSubscriptionStatus(companyId: CompanyId): Promise<SubscriptionStatus>
}
```

Module 5: Analytics & Reporting

Responsibilities:

- Dashboard metrics (revenue, occupancy, arrivals, departures)
- Historical trend analysis
- Forecasting and insights
- Custom report generation
- Business intelligence queries
- Data aggregation and caching

Database Tables:

- analytics_cache (new - for pre-computed metrics)
- materialized views (new - for performance)

API Endpoints:

- GET /analytics/dashboard (today's snapshot)
- GET /analytics/revenue (date range)
- GET /analytics/occupancy (date range)
- GET /analytics/trends (historical)
- POST /analytics/reports/generate (custom report)
- GET /analytics/forecast (predictive)

Domain Events Published:

- None (read-only module)

Domain Events Subscribed:

- ReservationConfirmed (update revenue metrics)
- GuestCheckedIn (update occupancy)
- GuestCheckedOut (update occupancy)
- PaymentConfirmed (update revenue)

Dependencies:

- Identity (tenant context)
- Booking (reservation data)
- Billing (payment data)
- Property Management (site data)

Module Interface:

```
interface AnalyticsModule {
  // Dashboard
  getDashboardMetrics(propertyId: PropertyId, date: Date): Promise<DashboardMetrics>

  // Revenue
  getRevenueReport(propertyId: PropertyId, range: DateRange): Promise<RevenueReport>
  getRevenueBySource(propertyId: PropertyId, range: DateRange):
```

```
Promise<RevenueBreakdown>

  // Occupancy
  getOccupancyRate(propertyId: PropertyId, range: DateRange):
Promise<OccupancyMetrics>
  getOccupancyTrends(propertyId: PropertyId, range: DateRange):
Promise<OccupancyTrend[]>

  // Insights
  getForecast(propertyId: PropertyId, horizon: number): Promise<Forecast>
  getRecommendations(propertyId: PropertyId): Promise<Recommendation[]>

  // Custom reports
  generateCustomReport(propertyId: PropertyId, config: ReportConfig): Promise<Report>
}
```

Module 6: Communications

Responsibilities:

- Email delivery (confirmations, reminders, receipts)
- SMS notifications (future)
- In-app notifications (future)
- Template management
- Delivery tracking
- Communication preferences

Database Tables:

- email_logs (new - for tracking)
- communication_preferences (new - for user prefs)

API Endpoints:

- POST /communications/email/send
- GET /communications/email/logs
- PATCH /communications/preferences
- GET /communications/templates

Domain Events Published:

- EmailSent
- EmailFailed
- SMSSent (future)

Domain Events Subscribed:

- ReservationConfirmed (send confirmation email)
- ReservationCancelled (send cancellation email)
- GuestCheckedIn (send welcome email)
- PaymentConfirmed (send receipt)
- PaymentFailed (send payment failure notice)

Dependencies:

- Identity (user contact info)
- Booking (reservation details)

Module Interface:

```
interface CommunicationsModule {
  // Email
  sendEmail(to: Email, template: TemplateId, data: TemplateData): Promise<void>
  sendReservationConfirmation(reservationId: ReservationId): Promise<void>
  sendCancellationNotice(reservationId: ReservationId): Promise<void>
  sendPaymentReceipt(paymentId: PaymentId): Promise<void>

  // SMS (future)
  sendSMS(to: Phone, message: string): Promise<void>

  // Preferences
  getPreferences(userId: UserId): Promise<CommunicationPreferences>
  updatePreferences(userId: UserId, prefs: Partial<CommunicationPreferences>):
Promise<void>

  // Tracking
  getEmailLogs(propertyId: PropertyId, range: DateRange): Promise<EmailLog[]>
}
```

Module 7: Shared Infrastructure

Responsibilities:

- Database connection pooling
- Caching (Redis - future)
- File storage (Supabase Storage)
- Logging and monitoring (Sentry)
- Feature flags
- Configuration management
- Health checks

Not a domain module - provides technical services to all modules

Module Interface:

```
interface InfrastructureModule {
  // Database
  getDbClient(role: 'anon' | 'authenticated' | 'service'): SupabaseClient

  // Cache (future)
  cacheGet(key: string): Promise<any>
  cacheSet(key: string, value: any, ttl: number): Promise<void>
  cacheInvalidate(pattern: string): Promise<void>

  // Storage
  uploadFile(bucket: string, path: string, file: File): Promise<string>
  deleteFile(bucket: string, path: string): Promise<void>

  // Monitoring
  logError(error: Error, context: ErrorContext): Promise<void>
  trackMetric(name: string, value: number): Promise<void>
```

```
  // Feature flags
  isFeatureEnabled(flag: FeatureFlag, context: FlagContext): Promise<boolean>

  // Health
  checkHealth(): Promise<HealthStatus>
}
```

---

3. Inter-Module Communication

3.1 Event-Driven Architecture

Internal Event Bus:

```
// Event Bus Interface
interface EventBus {
  publish<T extends DomainEvent>(event: T): Promise<void>
  subscribe<T extends DomainEvent>(
    eventType: string,
    handler: EventHandler<T>
  ): Subscription
}

// Domain Event Base
interface DomainEvent {
  eventId: EventId
  eventType: string
  aggregateId: string
  aggregateType: string
  occurredAt: Date
  payload: unknown
  metadata: {
    userId?: UserId
    tenantId?: CompanyId
    correlationId: CorrelationId
  }
}

// Example Event
interface ReservationConfirmed extends DomainEvent {
  eventType: 'ReservationConfirmed'
  aggregateType: 'Reservation'
  payload: {
    reservationId: ReservationId
    propertyId: PropertyId
    siteId: SiteId
    guestId: GuestId
    checkIn: Date
    checkOut: Date
    totalAmount: Money
  }
}
```

Event Store:

```
CREATE TABLE event_store (
  event_id UUID PRIMARY KEY,
  event_type VARCHAR(255) NOT NULL,
  aggregate_id UUID NOT NULL,
  aggregate_type VARCHAR(100) NOT NULL,
  occurred_at TIMESTAMPTZ NOT NULL,
  payload JSONB NOT NULL,
  metadata JSONB NOT NULL,
  processed BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_event_store_aggregate ON event_store(aggregate_type, aggregate_id);
CREATE INDEX idx_event_store_type ON event_store(event_type);
CREATE INDEX idx_event_store_occurred_at ON event_store(occurred_at);
CREATE INDEX idx_event_store_unprocessed ON event_store(processed) WHERE NOT
processed;
```

Event Handlers:

```
// Example: Analytics module subscribes to booking events
class RevenueMetricsHandler implements EventHandler<ReservationConfirmed> {
  async handle(event: ReservationConfirmed): Promise<void> {
    const { propertyId, totalAmount, occurredAt } = event.payload

    await this.analyticsRepository.updateRevenueMetrics({
      propertyId,
      date: occurredAt,
      revenue: totalAmount,
      bookingCount: 1
    })
  }
}

// Example: Communications module sends emails
class ReservationConfirmationEmailHandler implements
EventHandler<ReservationConfirmed> {
  async handle(event: ReservationConfirmed): Promise<void> {
    const reservation = await this.bookingModule.getReservation(
      event.payload.reservationId
    )

    await this.emailService.sendReservationConfirmation(reservation)
  }
}
```

3.2 Synchronous API Calls

For operations requiring immediate response:

```
// Example: Booking module queries Property module
class BookingService {
  constructor(
```

```
    private propertyModule: PropertyModule,
    private billingModule: BillingModule
  ) {}

  async createReservation(data: ReservationData): Promise<Reservation> {
    // 1. Validate site availability (sync call to Property module)
    const site = await this.propertyModule.getSite(data.siteId)
    const isAvailable = await this.propertyModule.checkAvailability(
      data.siteId,
      { start: data.checkIn, end: data.checkOut }
    )

    if (!isAvailable) {
      throw new SiteUnavailableError(data.siteId)
    }

    // 2. Calculate pricing (internal logic)
    const price = await this.calculatePrice(site, data)

    // 3. Create pending reservation
    const reservation = await this.repository.create({
      ...data,
      status: 'pending',
      totalAmount: price.total
    })

    // 4. Publish event (async)
    await this.eventBus.publish(new ReservationCreated(reservation))

    return reservation
  }
}
```

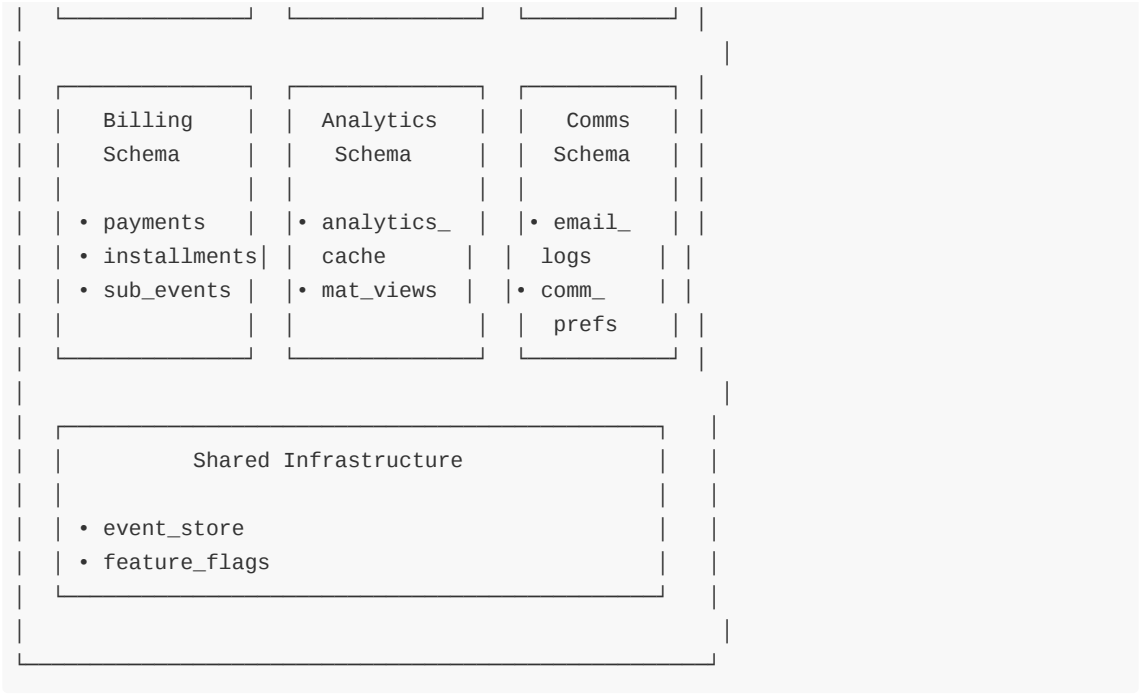
Guidelines:

- **Prefer events** for cross-module communication
- **Use sync calls** only when immediate response needed
- **Never circular dependencies** between modules
- **Define clear module interfaces** as contracts

3.3 Database Access Pattern

Shared Database with Logical Separation:

```
┌──────────────────────────────────────────────────────┐
│              PostgreSQL Database                     │
│                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌────────────┐  │
│  │   Identity   │  │   Booking    │  │  Property  │  │
│  │   Schema     │  │   Schema     │  │  Schema    │  │
│  │              │  │              │  │            │  │
│  │ • companies  │  │ • reservations│ │• properties│  │
│  │ • prop_staff │  │ • guests     │  │• sites     │  │
│  │              │  │ • res_actions│  │• templates │  │
```



Module Data Access Rules:

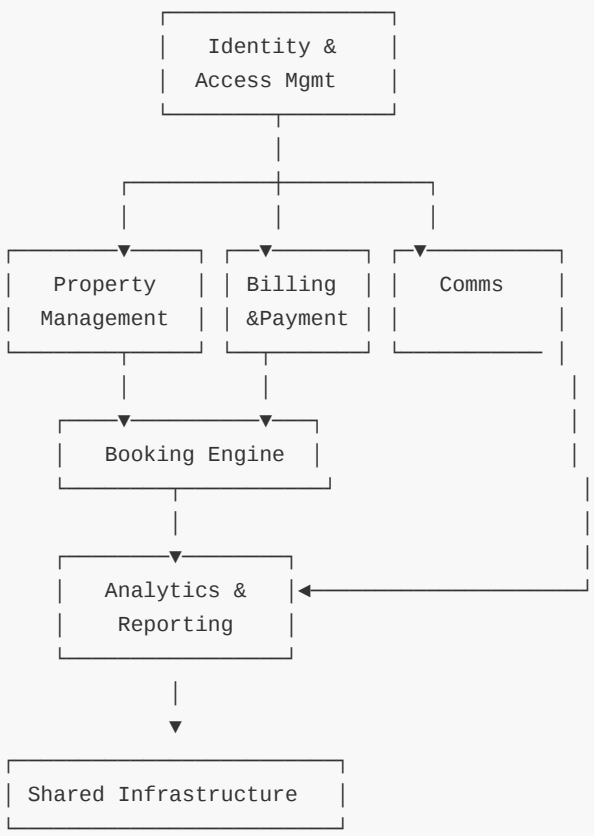
1. **Modules own their tables** - Only Identity module writes to `companies`
2. **Read-only cross-module access** - Booking can READ `sites`, cannot WRITE
3. **Use events for writes** - Booking publishes `GuestCheckedIn`, Property updates `site.status`
4. **RLS still enforced** - Multi-tenant isolation at database level

Repository Pattern per Module:

```
// Booking Module Repository
class ReservationRepository {
  constructor(private db: SupabaseClient) {}

  // Module owns reservations table - full CRUD
  async create(data: ReservationData): Promise<Reservation> {
    const { data: reservation } = await this.db
      .from('reservations')
      .insert(data)
      .select()
      .single()
    return reservation
  }

  // Can READ from sites (owned by Property module)
  async getSiteInfo(siteId: SiteId): Promise<Site> {
    const { data: site } = await this.db
      .from('sites')
      .select()
      .eq('id', siteId)
      .single()
    return site
```

```
  }

  // CANNOT WRITE to sites - must use PropertyModule API or event
  // ❌ async updateSiteStatus() - VIOLATION
}
```

4. Module Dependency Graph




```
Dependency Rules:
• Identity has NO dependencies (foundational)
• Property, Billing, Comms depend on Identity only
• Booking depends on Identity, Property, Billing
• Analytics depends on all domain modules (read-only)
• Infrastructure is used by all (not shown in graph)
```

5. API Gateway Design

Responsibilities:

- Route requests to appropriate module
- Enforce authentication & authorization
- Rate limiting & throttling
- Request/response transformation
- API versioning
- CORS handling
- Error normalization

Implementation:

```
// API Gateway Router
class ApiGateway {
  constructor(
    private identityModule: IdentityModule,
    private bookingModule: BookingModule,
    private propertyModule: PropertyModule,
    private billingModule: BillingModule,
    private analyticsModule: AnalyticsModule,
    private commsModule: CommunicationsModule
  ) {}

  async handleRequest(req: Request): Promise<Response> {
    // 1. Authenticate
    const auth = await this.identityModule.authenticate(req)

    // 2. Resolve tenant
    const tenant = await this.identityModule.resolveTenant(req)

    // 3. Authorize
    const permission = this.getRequiredPermission(req.url, req.method)
    const authorized = await this.identityModule.hasPermission(auth.userId,
permission)
    if (!authorized) {
      return new Response('Forbidden', { status: 403 })
    }

    // 4. Route to module
    const route = this.matchRoute(req.url)
    const handler = this.getModuleHandler(route.module, route.action)

    // 5. Execute
    try {
      const result = await handler(req, { auth, tenant })
      return this.formatResponse(result)
    } catch (error) {
      return this.handleError(error)
    }
  }

  private matchRoute(url: string): RouteMatch {
    // /api/v1/bookings/* → BookingModule
    // /api/v1/properties/* → PropertyModule
    // /api/v1/payments/* → BillingModule
    // etc.
  }
}
```

6. Testing Strategy

6.1 Module Testing

Unit Tests (per module):

```
// Test module in isolation with mocked dependencies
describe('BookingModule', () => {
  let bookingModule: BookingModule
  let mockPropertyModule: jest.Mocked<PropertyModule>
  let mockBillingModule: jest.Mocked<BillingModule>
  let mockEventBus: jest.Mocked<EventBus>

  beforeEach(() => {
    mockPropertyModule = createMockPropertyModule()
    mockBillingModule = createMockBillingModule()
    mockEventBus = createMockEventBus()

    bookingModule = new BookingModule(
      mockPropertyModule,
      mockBillingModule,
      mockEventBus
    )
  })

  it('should create reservation when site available', async () => {
    // Arrange
    mockPropertyModule.checkAvailability.mockResolvedValue(true)

    // Act
    const reservation = await bookingModule.createReservation({...})

    // Assert
    expect(reservation.status).toBe('pending')
    expect(mockEventBus.publish).toHaveBeenCalledWith(
      expect.objectContaining({ eventType: 'ReservationCreated' })
    )
  })
})
```

Integration Tests (module + database):

```
describe('BookingModule Integration', () => {
  let bookingModule: BookingModule
  let testDb: TestDatabase

  beforeEach(async () => {
    testDb = await setupTestDatabase()
    bookingModule = createBookingModule(testDb)
  })

  it('should persist reservation to database', async () => {
    const reservation = await bookingModule.createReservation({...})

    const persisted = await testDb
      .from('reservations')
```

```
      .select()
      .eq('id', reservation.id)
      .single()

    expect(persisted).toEqual(reservation)
  })
})
```

Contract Tests (module interfaces):

```
// Verify module implements its contract correctly
describe('BookingModule Contract', () => {
  it('should implement BookingModule interface', () => {
    const module = createBookingModule()

    // TypeScript ensures compile-time contract
    const _test: BookingModule = module // ✅ Compiles
  })

  it('should throw correct error types', async () => {
    const module = createBookingModule()

    await expect(
      module.createReservation({ siteId: 'invalid' })
    ).rejects.toThrow(SiteNotFoundError) // ✅ Contract-defined error
  })
})
```

6.2 End-to-End Tests

Cross-Module Workflows:

```
describe('Guest Booking E2E', () => {
  it('should complete full booking flow', async () => {
    // 1. Search availability (Booking + Property modules)
    const sites = await api.post('/booking/search-availability', {...})
    expect(sites).toHaveLength(5)

    // 2. Create reservation (Booking module)
    const reservation = await api.post('/booking/reserve', {...})
    expect(reservation.status).toBe('pending')

    // 3. Create payment (Billing module)
    const payment = await api.post('/booking/create-payment-intent', {...})

    // 4. Confirm payment (Billing module → publishes PaymentConfirmed event)
    await api.post('/guest/payment/confirm', { paymentIntentId: payment.id })

    // 5. Verify reservation confirmed (Booking module subscribed to event)
    await waitForEvent('ReservationConfirmed')
    const confirmed = await api.get(`/admin/reservations/${reservation.id}`)
    expect(confirmed.status).toBe('confirmed')
```

```
    // 6. Verify email sent (Comms module subscribed to event)
    const emails = await getEmailLogs()
    expect(emails).toContainEqual(
      expect.objectContaining({ template: 'reservation_confirmation' })
    )
  })
})
```

7. Deployment Strategy

Single Deployment Unit (Monolith):

```
┌─────────────────────────────────────────┐
│        Vercel Deployment                │
│                                         │
│  ┌──────────────────────────────────┐   │
│  │   Next.js Application            │   │
│  │                                  │   │
│  │  All 7 modules bundled together  │   │
│  │  Deployed as single unit         │   │
│  └──────────────────────────────────┘   │
│                                         │
│  • Fast deploys (single artifact)       │
│  • No distributed coordination          │
│  • Easier debugging                     │
│  • Can extract modules later            │
│                                         │
└─────────────────────────────────────────┘
```

Feature Flags for Gradual Rollout:

```
// Gradually enable new modular architecture
if (await featureFlags.isEnabled('modular_booking_v2', { tenantId })) {
  // New: Use modular BookingModule
  return bookingModuleV2.createReservation(data)
} else {
  // Old: Use existing API route logic
  return legacyCreateReservation(data)
}
```

Module Extraction (Future):

```
Option 1: Extract to Microservice
┌──────────────┐      ┌──────────────┐
│  Monolith    │      │  Billing     │
│              │◄────►│  Service     │
│ - Booking    │      │              │
│ - Property   │      │ (Standalone) │
│ - Analytics  │      │              │
└──────────────┘      └──────────────┘

Option 2: Keep as Modular Monolith
┌──────────────────────────────────────┐
```

```
|     Single Deployment           |
|                                 |
|  All modules remain bundled     |
|  Benefits without complexity    |
|                                 |
└─────────────────────────────────┘
```

Transition Plan

Phase 1: Foundation (Months 1-2)

Goal: Establish modular architecture without breaking existing functionality

1.1 Event Store Implementation

Tasks:

- [ ] Create `event_store` table
- [ ] Implement EventBus interface
- [ ] Create base DomainEvent types
- [ ] Add event publishing infrastructure
- [ ] Implement event handler registration
- [ ] Add idempotency protection (event_id uniqueness)

Deliverables:

```
// event-bus.ts
export class PostgresEventBus implements EventBus {
  async publish<T extends DomainEvent>(event: T): Promise<void> {
    await this.db.from('event_store').insert({
      event_id: event.eventId,
      event_type: event.eventType,
      aggregate_id: event.aggregateId,
      aggregate_type: event.aggregateType,
      occurred_at: event.occurredAt,
      payload: event.payload,
      metadata: event.metadata
    })

    // Trigger handlers asynchronously
    await this.triggerHandlers(event)
  }

  subscribe<T extends DomainEvent>(
    eventType: string,
    handler: EventHandler<T>
  ): Subscription {
    this.handlers.set(eventType, handler)
    return { unsubscribe: () => this.handlers.delete(eventType) }
  }
}
```

Migration Steps:

1. Run database migration to create `event_store`
2. Deploy EventBus implementation (no consumers yet)
3. Add event publishing to one API route (pilot)
4. Verify events appear in database
5. Add first event handler (idempotent)
6. Verify handler execution

Risks & Mitigations:

- **Risk:** Event handling failures could break flows
  - **Mitigation:** All handlers must be idempotent, implement retry logic
- **Risk:** Performance impact from event storage
  - **Mitigation:** Index event_store properly, async processing

Success Criteria:

- [ ] Event store operational
- [ ] Can publish and subscribe to events
- [ ] Zero production incidents
- [ ] Pilot event handler working correctly

1.2 Module Directory Structure

Tasks:

- [ ] Create `/modules` directory
- [ ] Define module interfaces for all 7 modules
- [ ] Create module factory pattern
- [ ] Implement dependency injection container

New Directory Structure:

```
/modules
  /identity
    /domain       - Domain models (User, Company, Tenant)
    /application  - Use cases (Login, Register, ResolveTenant)
    /infrastructure - Supabase implementation
    /api          - API route handlers
    index.ts      - Module interface export

  /booking
    /domain       - Reservation, Guest, Pricing
    /application  - CreateReservation, CheckIn, CheckOut
    /infrastructure - Database repositories
    /api          - API route handlers
    index.ts

  /property
    /domain       - Property, Site, Configuration
    /application  - CreateProperty, UpdateSite
    /infrastructure - Repositories
```

```
    /api
    index.ts

  /billing
    /domain       - Payment, Subscription, Invoice
    /application  - ProcessPayment, HandleWebhook
    /infrastructure - Stripe integration
    /api
    index.ts

  /analytics
    /domain       - Metric, Report, Forecast
    /application  - CalculateRevenue, GenerateReport
    /infrastructure - Analytics queries
    /api
    index.ts

  /communications
    /domain       - Email, SMS, Notification
    /application  - SendEmail, SendSMS
    /infrastructure - Resend integration
    /api
    index.ts

  /shared
    /infrastructure - EventBus, Database, Cache, Storage
```

Module Factory:

```typescript
// modules/module-factory.ts
export class ModuleFactory {
  private static modules: Map<string, any> = new Map()

  static createIdentityModule(deps: IdentityDependencies): IdentityModule {
    if (!this.modules.has('identity')) {
      this.modules.set('identity', new IdentityModuleImpl(deps))
    }
    return this.modules.get('identity')
  }

  static createBookingModule(deps: BookingDependencies): BookingModule {
    if (!this.modules.has('booking')) {
      const identityModule = this.createIdentityModule(deps.identity)
      const propertyModule = this.createPropertyModule(deps.property)
      const billingModule = this.createBillingModule(deps.billing)

      this.modules.set('booking', new BookingModuleImpl({
        identityModule,
        propertyModule,
        billingModule,
        eventBus: deps.eventBus,
        db: deps.db
```

```
      }))
    }
    return this.modules.get('booking')
  }

  // ... other modules
}
```

Migration Steps:

1. Create `/modules` directory structure
2. Move existing `/lib/booking` logic to `/modules/booking`
3. Extract interfaces for each module
4. Implement factory pattern
5. Update imports in existing code (gradual migration)

Success Criteria:

- [ ] Module structure created
- [ ] All modules have defined interfaces
- [ ] Factory pattern working
- [ ] Zero breaking changes to existing API

1.3 Webhook Idempotency Fix (CRITICAL)

Tasks:

- [ ] Add `stripe_event_id` unique constraint to `subscription_events`
- [ ] Implement event deduplication logic
- [ ] Add database transaction support
- [ ] Update webhook handlers to use idempotency

Implementation:

```
// webhook-handler.ts
export async function handleStripeWebhook(event: Stripe.Event): Promise<void> {
  // 1. Check if already processed
  const exists = await db
    .from('subscription_events')
    .select('id')
    .eq('stripe_event_id', event.id)
    .single()

  if (exists) {
    console.log(`Event ${event.id} already processed, skipping`)
    return // Idempotent - safe to ignore
  }

  // 2. Start transaction
  const { data, error } = await db.rpc('process_webhook_event', {
    event_id: event.id,
    event_type: event.type,
```

```
    event_data: event.data
  })

  if (error) {
    throw new WebhookProcessingError(error)
  }

  // 3. Publish domain event (if applicable)
  if (event.type === 'customer.subscription.created') {
    await eventBus.publish(new SubscriptionCreated({
      companyId: data.company_id,
      subscriptionId: event.data.object.id
    }))
  }
}
```

Database Function:

```
CREATE OR REPLACE FUNCTION process_webhook_event(
  event_id TEXT,
  event_type TEXT,
  event_data JSONB
) RETURNS JSONB AS $$
DECLARE
  result JSONB;
BEGIN
  -- Insert event (unique constraint prevents duplicates)
  INSERT INTO subscription_events (stripe_event_id, event_type, event_data)
  VALUES (event_id, event_type, event_data)
  ON CONFLICT (stripe_event_id) DO NOTHING;

  -- Process based on event type
  CASE event_type
    WHEN 'checkout.session.completed' THEN
      result := process_checkout_completed(event_data);
    WHEN 'customer.subscription.created' THEN
      result := process_subscription_created(event_data);
    -- ... other cases
  END CASE;

  RETURN result;
END;
$$ LANGUAGE plpgsql;
```

Success Criteria:

- [ ] No duplicate companies from webhook retries
- [ ] Transaction rollback on failure
- [ ] 100% idempotent webhook processing

Phase 2: Core Module Extraction (Months 3-4)

Goal: Extract Identity and Booking modules as pilots

2.1 Identity Module

Tasks:

- [ ] Move authentication logic to IdentityModule
- [ ] Implement module interface
- [ ] Update middleware to use module
- [ ] Add contract tests
- [ ] Feature flag for gradual rollout

Migration:

```
// Before (lib/middleware/auth.ts)
export async function authMiddleware(req: Request) {
  const supabase = createClient()
  const { data: { user } } = await supabase.auth.getUser()
  // ... logic
}

// After (modules/identity/application/authenticate.ts)
export class AuthenticateUseCase {
  constructor(private supabaseClient: SupabaseClient) {}

  async execute(req: Request): Promise<AuthContext> {
    const { data: { user } } = await this.supabaseClient.auth.getUser()

    if (!user) {
      throw new NotAuthenticatedError()
    }

    return {
      userId: user.id as UserId,
      email: user.email,
      emailVerified: !!user.email_confirmed_at
    }
  }
}

// Middleware now uses module
const identityModule = ModuleFactory.createIdentityModule()
const authContext = await identityModule.authenticate(req)
```

Success Criteria:

- [ ] All authentication goes through IdentityModule
- [ ] Middleware uses module interface
- [ ] Zero regressions in auth flow
- [ ] Contract tests pass

2.2 Booking Module

Tasks:

- [ ] Extract reservation logic to BookingModule
- [ ] Implement availability checking
- [ ] Implement pricing calculation
- [ ] Add domain events (ReservationCreated, etc.)
- [ ] Update API routes to use module
- [ ] Add event handlers for site status updates

Event-Driven Site Status:

```
// Before: Direct database update
await db.from('sites').update({ status: 'occupied' }).eq('id', siteId)

// After: Event-driven
// 1. Booking module publishes event
await eventBus.publish(new GuestCheckedIn({
  reservationId,
  siteId,
  checkInTime: new Date()
}))

// 2. Property module handles event
class GuestCheckedInHandler implements EventHandler<GuestCheckedIn> {
  async handle(event: GuestCheckedIn): Promise<void> {
    await this.propertyModule.updateSiteStatus(
      event.payload.siteId,
      'occupied'
    )
  }
}
```

Success Criteria:

- [ ] All booking operations through BookingModule
- [ ] Events published for state changes
- [ ] Cross-module updates via events
- [ ] Zero booking flow regressions

Phase 3: Remaining Modules (Months 5-6)

Goal: Complete module extraction for all 7 modules

3.1 Property Management Module

Tasks:

- [ ] Extract property/site CRUD
- [ ] Extract configuration system

- [ ] Implement event handlers (GuestCheckedIn, GuestCheckedOut)
- [ ] Migrate onboarding wizard

Success Criteria:

- [ ] Property operations modularized
- [ ] Event-driven site status updates working
- [ ] Onboarding wizard using module

3.2 Billing Module

Tasks:

- [ ] Extract Stripe integration
- [ ] Implement webhook idempotency (from Phase 1)
- [ ] Add payment domain events
- [ ] Migrate subscription management

Success Criteria:

- [ ] All payments through BillingModule
- [ ] Webhook processing 100% idempotent
- [ ] Payment events published correctly

3.3 Analytics Module

Tasks:

- [ ] Create analytics materialized views
- [ ] Implement caching layer
- [ ] Build dashboard query service
- [ ] Subscribe to relevant events for cache updates

New Tables:

```
CREATE MATERIALIZED VIEW analytics_daily_revenue AS
SELECT
  property_id,
  DATE(occurred_at) as date,
  SUM(total_amount) as total_revenue,
  COUNT(*) as booking_count
FROM reservations
WHERE status = 'confirmed'
GROUP BY property_id, DATE(occurred_at);

CREATE TABLE analytics_cache (
  cache_key VARCHAR(255) PRIMARY KEY,
  property_id UUID NOT NULL,
  data JSONB NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
```

```
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

Success Criteria:

- [ ] Dashboard queries use cached data
- [ ] Real-time updates via events
- [ ] Query performance < 100ms

3.4 Communications Module

Tasks:

- [ ] Extract email sending logic
- [ ] Implement template management
- [ ] Add email logging
- [ ] Subscribe to booking/payment events

Success Criteria:

- [ ] All emails through CommunicationsModule
- [ ] Event-driven email sending
- [ ] Email delivery tracking working

Phase 4: Optimization & Polish (Months 7-9)

Goal: Optimize performance, add monitoring, comprehensive testing

4.1 Performance Optimization

Tasks:

- [ ] Implement Redis caching layer
- [ ] Add database query optimization
- [ ] Implement connection pooling
- [ ] Add CDN for static assets

Caching Strategy:

```
interface CacheService {
  get<T>(key: string): Promise<T | null>
  set<T>(key: string, value: T, ttl: number): Promise<void>
  invalidate(pattern: string): Promise<void>
}

// Example: Cache subscription status
const cacheKey = `subscription:${companyId}`
let subscriptionStatus = await cache.get<SubscriptionStatus>(cacheKey)

if (!subscriptionStatus) {
  subscriptionStatus = await db.getSubscriptionStatus(companyId)
```

```
  await cache.set(cacheKey, subscriptionStatus, 300) // 5 min TTL
}
```

Success Criteria:

- [ ] Middleware queries reduced by 80%
- [ ] Average response time < 200ms
- [ ] Database load reduced by 50%

4.2 Comprehensive Testing

Tasks:

- [ ] Add contract tests for all modules
- [ ] Increase integration test coverage to 90%
- [ ] Add E2E tests for critical flows
- [ ] Implement chaos testing

Contract Test Example:

```
describe('BookingModule Contract', () => {
  const contractTests = createContractTests<BookingModule>()

  contractTests.testMethod('createReservation', {
    given: 'valid reservation data',
    when: 'createReservation is called',
    then: 'should return Reservation object',
    and: 'should publish ReservationCreated event'
  })

  contractTests.testMethod('createReservation', {
    given: 'site not available',
    when: 'createReservation is called',
    then: 'should throw SiteUnavailableError'
  })
})
```

Success Criteria:

- [ ] 90%+ code coverage
- [ ] All modules have contract tests
- [ ] E2E tests for top 10 user journeys
- [ ] Zero critical bugs in production

4.3 Monitoring & Observability

Tasks:

- [ ] Add module-level metrics
- [ ] Implement distributed tracing
- [ ] Add business metrics dashboard

- [ ] Create alerting rules

Metrics:

```
// Module performance metrics
metrics.track('booking.reservation.created', {
  duration: executionTime,
  status: 'success',
  propertyId,
  siteType
})

// Business metrics
metrics.track('revenue.booking.confirmed', {
  amount: totalAmount,
  propertyId,
  bookingType
})

// Error metrics
metrics.track('error.booking.failed', {
  errorType: error.constructor.name,
  module: 'booking'
})
```

Success Criteria:

- [ ] Real-time module health dashboard
- [ ] Alerts for critical errors
- [ ] Business KPIs tracked
- [ ] MTTR < 15 minutes

Phase 5: Future Enhancements (Months 10+)

Goal: Advanced features and potential microservices extraction

5.1 Advanced Features

Tasks:

- [ ] Multi-company portfolio management
- [ ] Staff role-based access control
- [ ] Guest self-service portal
- [ ] Mobile app API
- [ ] Third-party integrations

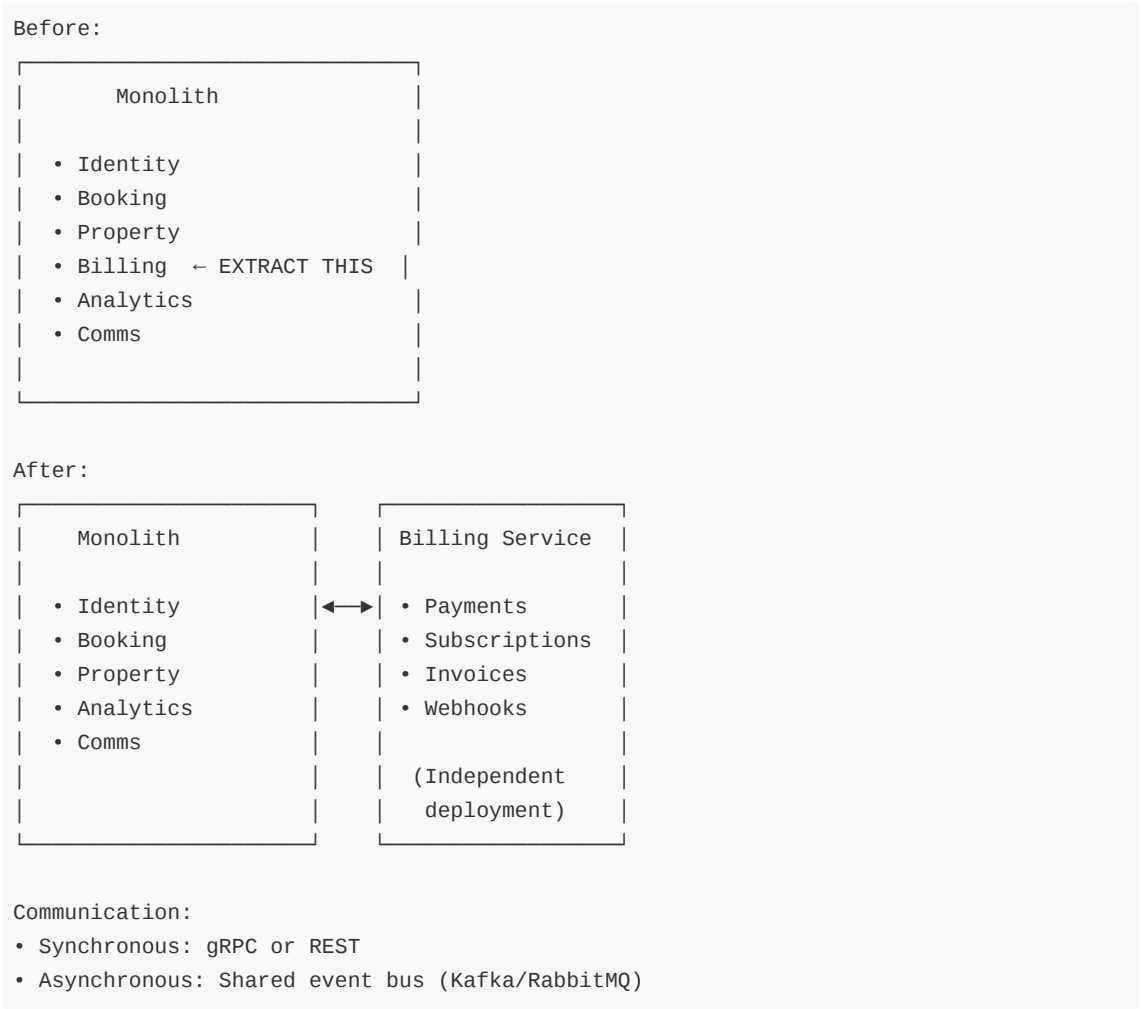
5.2 Microservices Extraction (Optional)

When to Extract:

- Module becomes performance bottleneck
- Need independent scaling (e.g., Analytics during reporting)

- Want independent deployment cycles
- Different technology requirements

Example: Extract Billing to Microservice:



Migration Steps:

1. Ensure module fully isolated (no leaky abstractions)
2. Add service-to-service authentication
3. Deploy as separate service
4. Update API Gateway to route billing requests
5. Monitor performance and rollback if needed

Technical Decision Log

Decision 1: Event Bus Implementation

Context: Need asynchronous inter-module communication

Options:

1. In-process event bus (function calls)
2. Database-backed event store
3. External message queue (RabbitMQ, Kafka)

Decision: Database-backed event store (PostgreSQL)

Rationale:

- Single deployment unit (no additional infrastructure)
- Transactional guarantees with database
- Event sourcing for audit trail
- Can migrate to external queue later if needed
- Lower operational complexity

Trade-offs:

- Slightly higher latency than in-process
- Database write for every event
- Limited throughput (acceptable for current scale)

Decision 2: Shared Database vs Database per Module

Context: Module data isolation strategy

Options:

1. Shared database with logical separation
2. Database per module
3. Shared database with schemas per module

Decision: Shared database with logical separation

Rationale:

- Simpler deployment and operations
- No distributed transactions needed
- Can still enforce boundaries via code
- RLS provides multi-tenant isolation
- Can split later if needed

Trade-offs:

- Harder to enforce strict boundaries
- Potential for accidental coupling
- Module ownership must be disciplined

Decision 3: Synchronous vs Asynchronous Module Communication

Context: How modules communicate

Options:

1. All synchronous (function calls)
2. All asynchronous (events only)
3. Hybrid (sync for queries, async for commands)

Decision: Hybrid approach

Rationale:

- Immediate feedback for queries (getReservation)
- Eventual consistency for state changes (reservation confirmed)

- Better user experience (sync) + decoupling (async)
- Realistic for business requirements

Trade-offs:

- More complex than pure approach
- Need clear guidelines on when to use each
- Developers must understand both patterns

Decision 4: Module Extraction Timeline

Context: When to extract modules from monolith

Options:

1. Extract all modules immediately (big bang)
2. Gradual extraction with feature flags
3. Never extract (stay modular monolith)

Decision: Gradual extraction with feature flags

Rationale:

- Lower risk (can rollback)
- Learn from early modules
- Maintain production stability
- Validate architecture incrementally

Trade-offs:

- Slower overall migration
- Temporary code duplication
- Feature flag management overhead

Appendices

Appendix A: Module Interface Reference

See detailed interface definitions in Future State section for all 7 modules.

Appendix B: Event Catalog

Identity Module Events:

- UserRegistered
- UserLoggedIn
- TenantResolved
- StaffMemberAdded

Booking Module Events:

- ReservationCreated
- ReservationConfirmed
- ReservationCancelled
- GuestCheckedIn
- GuestCheckedOut
- ReservationExtended

- ReservationRenewed

Property Module Events:

- PropertyCreated
- PropertyUpdated
- SiteCreated
- SiteUpdated
- SiteStatusChanged
- ConfigurationUpdated
- OnboardingCompleted

Billing Module Events:

- PaymentConfirmed
- PaymentFailed
- SubscriptionCreated
- SubscriptionUpdated
- SubscriptionCancelled
- InvoicePaid
- RefundIssued

Communications Module Events:

- EmailSent
- EmailFailed

Appendix C: Migration Checklist

Phase 1: Foundation

- [ ] Event store table created
- [ ] EventBus implemented
- [ ] Module directory structure created
- [ ] Module interfaces defined
- [ ] Webhook idempotency fixed
- [ ] Database transactions added

Phase 2: Core Modules

- [ ] Identity module extracted
- [ ] Booking module extracted
- [ ] Event handlers implemented
- [ ] Contract tests added

Phase 3: Remaining Modules

- [ ] Property module extracted
- [ ] Billing module extracted
- [ ] Analytics module created
- [ ] Communications module created

Phase 4: Optimization

- [ ] Redis caching implemented

- [ ] Performance benchmarks met
- [ ] Test coverage > 90%
- [ ] Monitoring dashboard live

Appendix D: Risk Register

Risk	Impact	Probability	Mitigation
Event bus performance	High	Medium	Index event_store, async processing
Module coupling	Medium	High	Enforce interfaces, contract tests
Breaking changes	High	Medium	Feature flags, gradual rollout
Developer confusion	Medium	High	Documentation, training, code reviews
Production incidents	High	Low	Comprehensive testing, monitoring
Timeline delays	Medium	Medium	Prioritize ruthlessly, MVP approach

---

Appendix E: Glossary

Bounded Context: A logical boundary within which a domain model is defined and applicable.

Domain Event: A record of something that happened in the domain that is of interest to the business.

Event Bus: Infrastructure for publishing and subscribing to domain events.

Event Sourcing: Storing state changes as a sequence of events rather than current state.

Modular Monolith: Single deployment unit with well-defined internal module boundaries.

Module: Self-contained unit with clear responsibilities, interface, and dependencies.

Service: Standalone deployable unit (microservice).

End of System Design Document

Document Maintainer: Technical Leadership **Review Cycle:** Quarterly or after major architectural changes **Last Updated:** November 5, 2025 **Next Review:** February 5, 2026